# 15. ROS 2 Time-Related API

## 1. Introduction to Time-Related APIs

The time-related APIs in ROS 2 include Rate, Time, Duration, and operations between Time and Duration, which are explained separately below.

- First, create a package to store the relevant program files

```
ros2 pkg create learning_time --build-type ament_python --dependencies rclpy
```

## 2. create_rate

ROS 2 also provides the create_rate function, a tool used for **controlling the loop execution frequency**. Its core function is to make a piece of code execute **periodically at a fixed frequency**. `Rate` guarantees the stability of the loop execution frequency by controlling the "sleep time" of the loop. **Remember** that `Rate` generally cannot be used directly in the main thread, otherwise it will permanently block callback events. It is generally only used in programs with multi-threaded callbacks or in child threads.

Specific working principle:

1. Record the time when each loop starts;
2. Execute the code inside the loop;
3. Calculate the difference between the actual elapsed time of the current loop and the "target interval time";
4. Automatically sleep for the corresponding difference time, ensuring that the interval from the start of one loop to the start of the next loop is strictly equal to the "target interval" (such as 100 milliseconds).

Although both `Rate` and `Timer` can implement periodic execution, their applicable scenarios are different:

| **Feature** | `Rate` | `Timer` |
| --- | --- | --- |
| Implementation method | Based on active sleep within the loop (blocking the current thread) | Based on callback functions (non-blocking, triggered by the ROS 2 event loop) |
| Applicable scenarios | Suitable for loops that need to execute at a fixed frequency **within the same thread** (such as the main control logic) | Suitable for periodic tasks that need **asynchronous execution** (without blocking the main thread) |
| Flexibility | The flow can be controlled directly within the loop (such as exiting with break) | Callback execution needs to be controlled through flags or other methods |

- The following is the basic usage of `Rate`, implementing a loop that executes 2 times per second:

- Create a new file rate_demo.py in the package

```
import rclpy
from rclpy.node import Node
import threading

class RateExampleNode(Node):
    def __init__(self):
        super().__init__("rate_example_node")
        self.get_logger().info("Rate example node started")

    def run_loop(self):
        # Use the node's create_rate() to create a 2Hz Rate
        rate = self.create_rate(2)

        count = 0
        try:
            while rclpy.ok():
                self.get_logger().info(f"Loop executed {count} times")
                count += 1
                rate.sleep()  # Sleep until the next cycle (0.5 seconds)
        except KeyboardInterrupt:
            self.get_logger().info("Loop interrupted")

def main(args=None):
    rclpy.init(args=args)
    node = RateExampleNode()

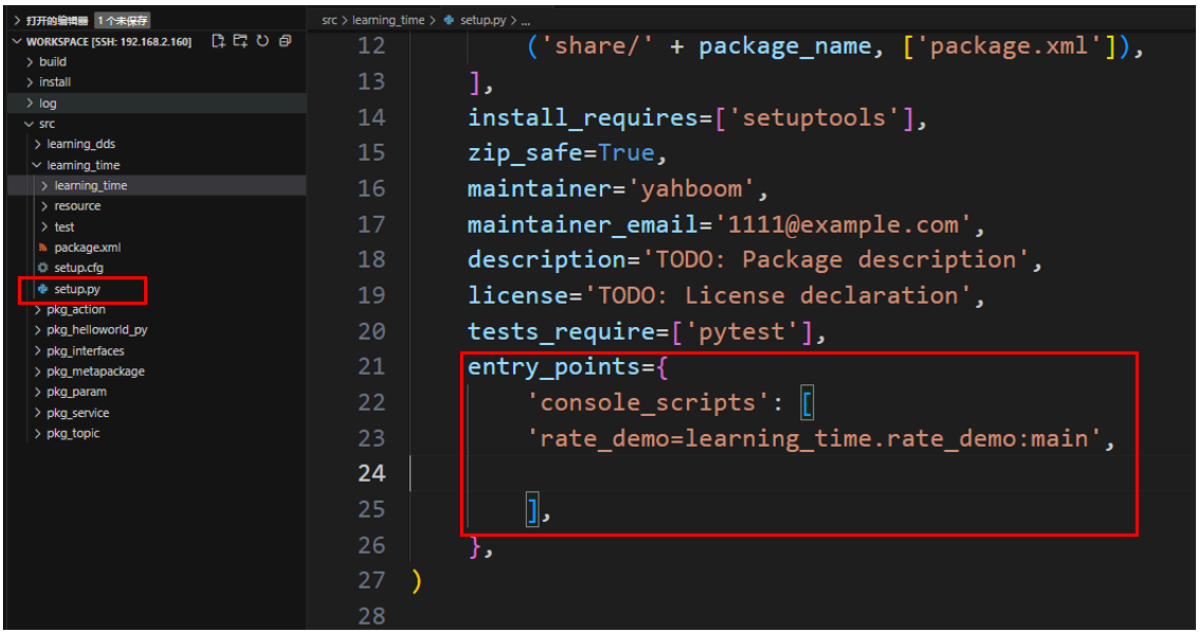
    # Create a thread to run the loop (avoid blocking the main thread)
    loop_thread = threading.Thread(target=node.run_loop)
    loop_thread.start()

    # The main thread executes spin to keep the ROS 2 node running
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        loop_thread.join()  # Wait for the thread to end
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()
```

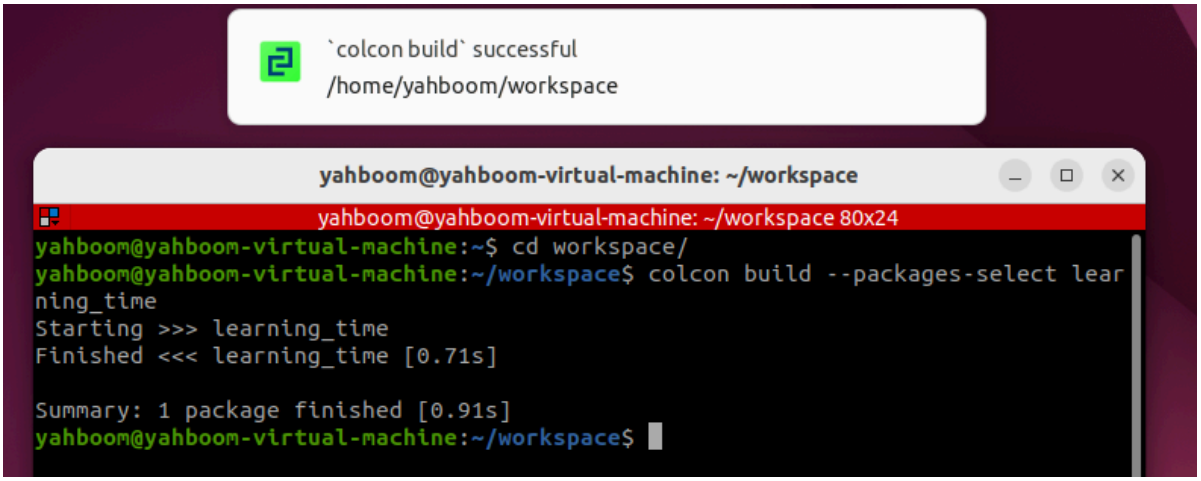
- Configure the corresponding setup.py configuration file, and add the following to console_scripts

```
'rate_demo=learning_time.rate_demo:main'
```




- Compile the package

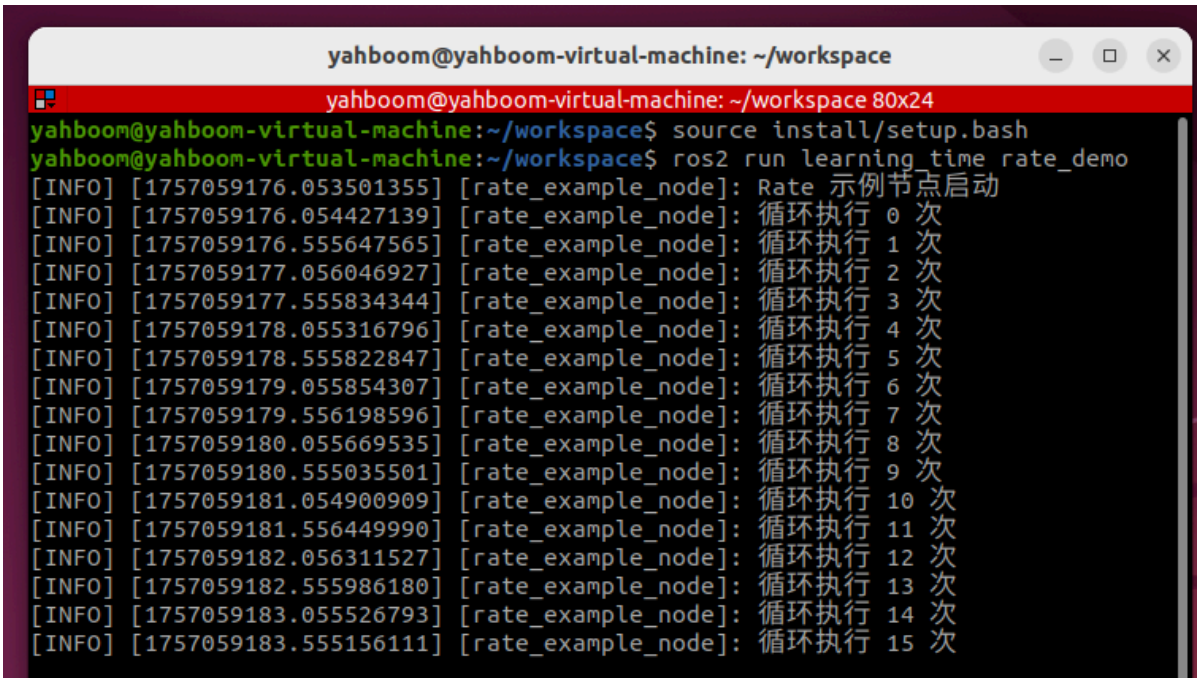
```
colcon build --packages-select learning_time
```




- Refresh the workspace environment and run the node

```
source ./install/setup.bash
```

```
ros2 run learning_time rate_demo
```




# 3. Timer Application

- Timer is used to create a timer that triggers at a fixed time to execute some periodic tasks
- Example: Create two timers. One executes once every 1 second and prints the number of executions, and the other executes once every 0.5 seconds and prints the current time
- Create a new program file Timer_demo.py

```
import rclpy
from rclpy.node import Node

class TimerDemoNode(Node):
    def __init__(self):
        super().__init__('timer_demo_node')

        # Counter, used to demonstrate the number of timer executions
        self.counter = 0

        # Create a timer: execute the callback function once every 1 second
        self.timer = self.create_timer(1.0, self.timer_callback)

        # Create a faster timer: execute once every 0.5 seconds
        self.fast_timer = self.create_timer(0.5, self.fast_timer_callback)

        self.get_logger().info("Timer node has started")

    def timer_callback(self):
        """1-second timer callback function"""
        self.counter += 1
        current_time = self.get_clock().now()

        # Print the current time and the counter value
        self.get_logger().info(
            f"[1-second timer] Execution {self.counter}, current time:
{current_time.seconds_nanoseconds()}"
        )
```

```
    def fast_timer_callback(self):
        """0.5-second timer callback function"""
        # Print the current timestamp (nanoseconds)
        self.get_logger().info(
            f"[0.5-second timer] Current timestamp:
{self.get_clock().now().nanoseconds}"
        )

def main(args=None):
    # Initialize ROS 2
    rclpy.init(args=args)

    # Create the node
    node = TimerDemoNode()

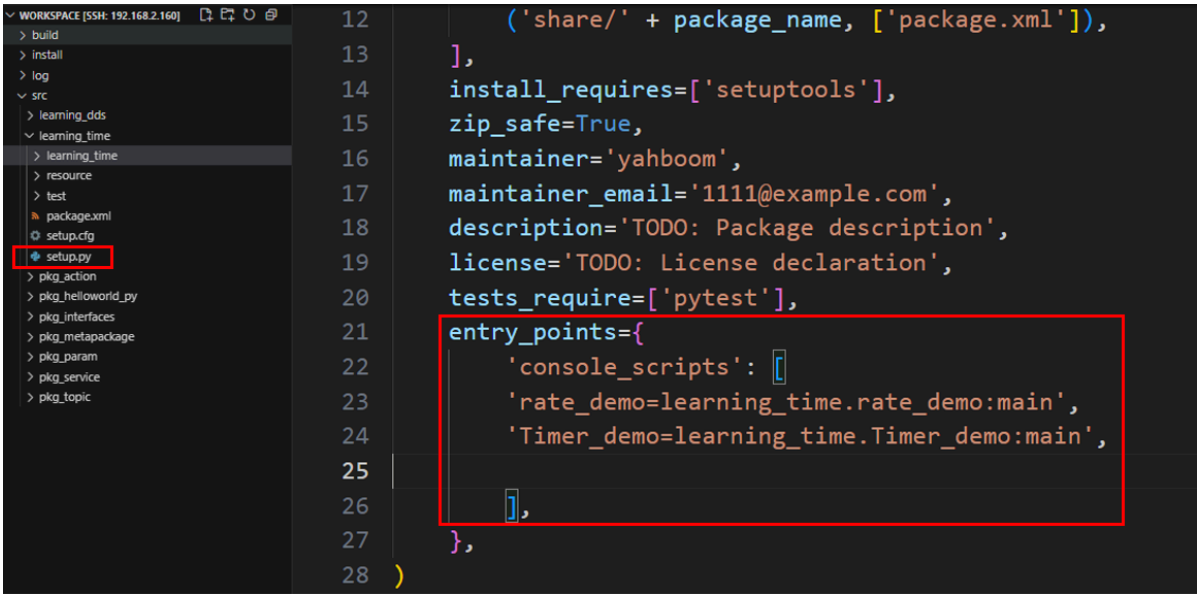
    # Run the node
    rclpy.spin(node)

    # Shut down ROS 2
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

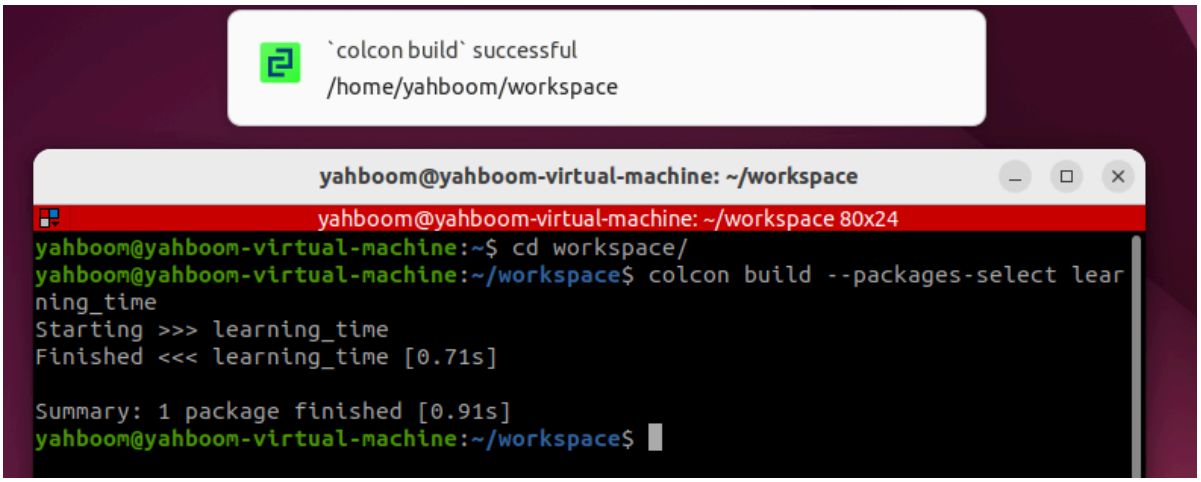
- Configure the corresponding setup.py configuration file, and add the following to console_scripts

```
'Timer_demo=learning_time.Timer_demo:main'
```




- Compile the package

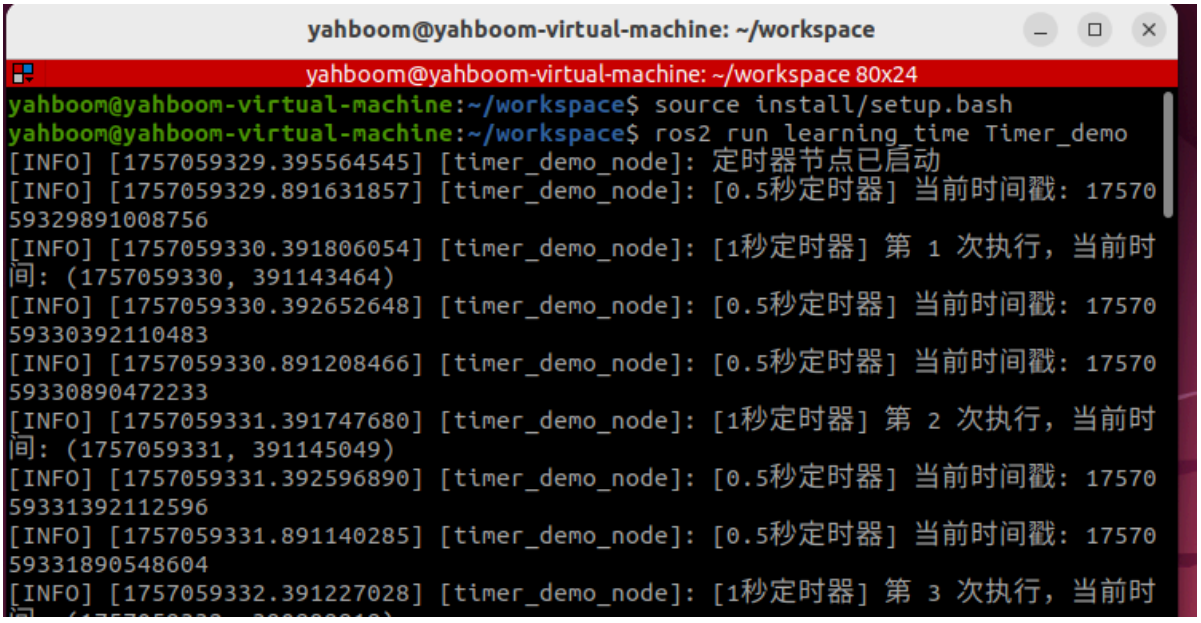
```
colcon build --packages-select learning_time
```




- Refresh the workspace environment and run the node

```
source ./install/setup.bash
```

```
ros2 run learning_time Timer_demo
```




- The timer fires at the configured interval and prints the corresponding log in its callback function

# 4. Getting the Current Time with get_clock

- The get_clock function can be used to obtain the clock object, and then the () method is used to obtain the current time
- Create a new program file get_clock_demo.py and fill in the following example program:

```
import rclpy
from rclpy.node import Node
from rclpy.time import Time

class TimeExampleNode(Node):
    def __init__(self):
        super().__init__("time_example_node")
```

```
        # Get the clock object of the node (the system clock is used by default)
        self.clock = self.get_clock()

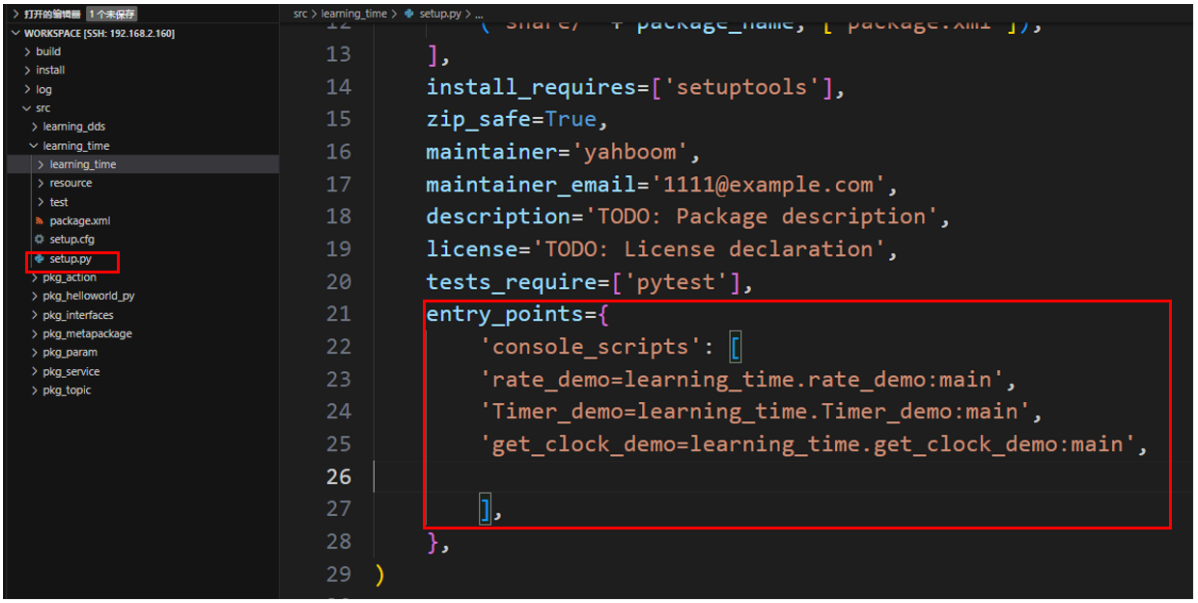
        # Get the current time (returns a Time object)
        current_time = self.clock.now()
        self.get_logger().info(f"Current time: {current_time}")

def main(args=None):
    rclpy.init(args=args)
    node = TimeExampleNode()
    rclpy.spin_once(node)  # Run the node once
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

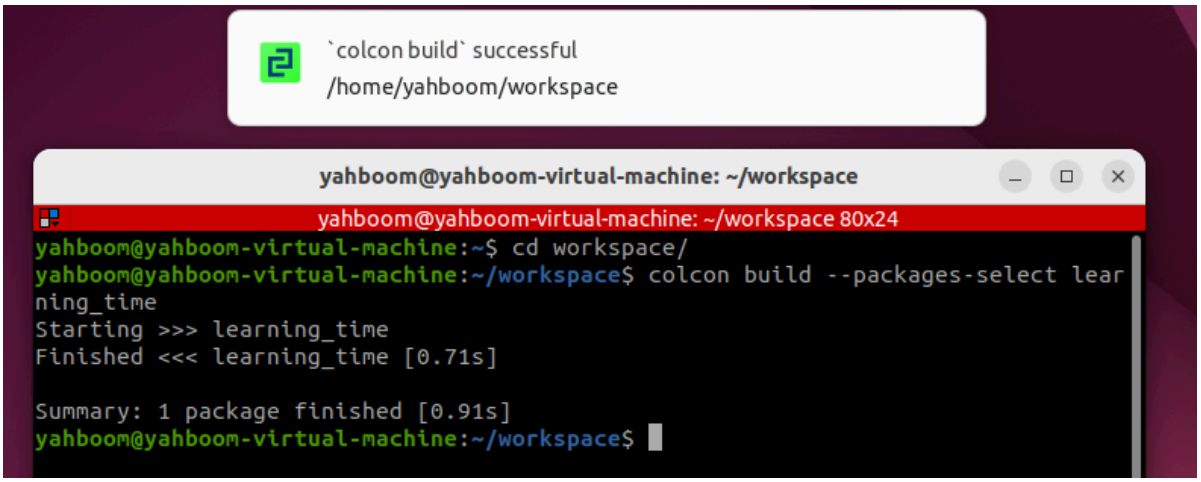
- Configure the corresponding setup.py configuration file, and add the following to console_scripts

```
'get_clock_demo=learning_time.get_clock_demo:main'
```




- Compile the package

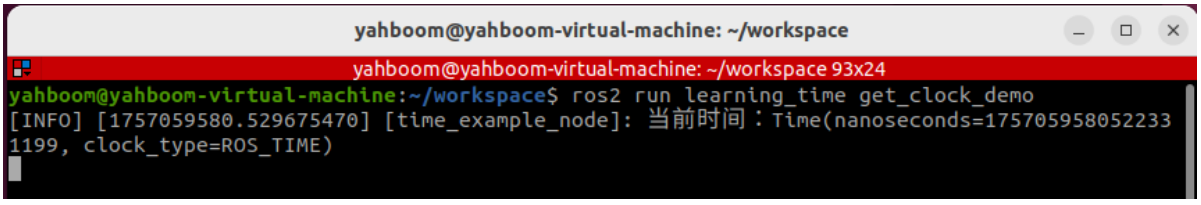
```
colcon build --packages-select learning_time
```




- Refresh the workspace environment and run the node

```
source ./install/setup.bash
```

```
ros2 run learning_time get_clock_demo
```




# 5. Time and Duration

- The `Time` class is used in ros to represent a specific **time point** (such as "2023-10-01 12:00:00"), and is usually used to mark the **moment** when an event occurs.
- The `Duration` class represents the **interval** between two time points (such as "5 seconds"), and is used to calculate the time difference or delay.
- **Example:** Time and Duration application
- Create a new program file TimeDuration_demo.py

```
import rclpy
from rclpy.time import Time
from rclpy.duration import Duration

def main():
    rclpy.init()
    node = rclpy.create_node("time_opt_node")

    # Usage of the time class, creating a 'time point, moment'
    time1 = Time(seconds=10)
    time2 = Time(seconds=4)
    # Usage of the Duration class, creating a 'duration, a period of time'
    duration1 = Duration(seconds=3)
    duration2 = Duration(seconds=5)

    # Moments can be compared
    node.get_logger().info("time1 >= time2 ? %d" % (time1 >= time2))
    node.get_logger().info("time1 < time2 ? %d" % (time1 < time2))
```

```
    # Time periods and moments can be operated on mathematically
    t3 = time1 + duration1
    t4 = time1 - time2
    t5 = time1 - duration1

    node.get_logger().info("t3 = %d" % t3.nanoseconds)
    node.get_logger().info("t4 = %d" % t4.nanoseconds)
    node.get_logger().info("t5 = %d" % t5.nanoseconds)

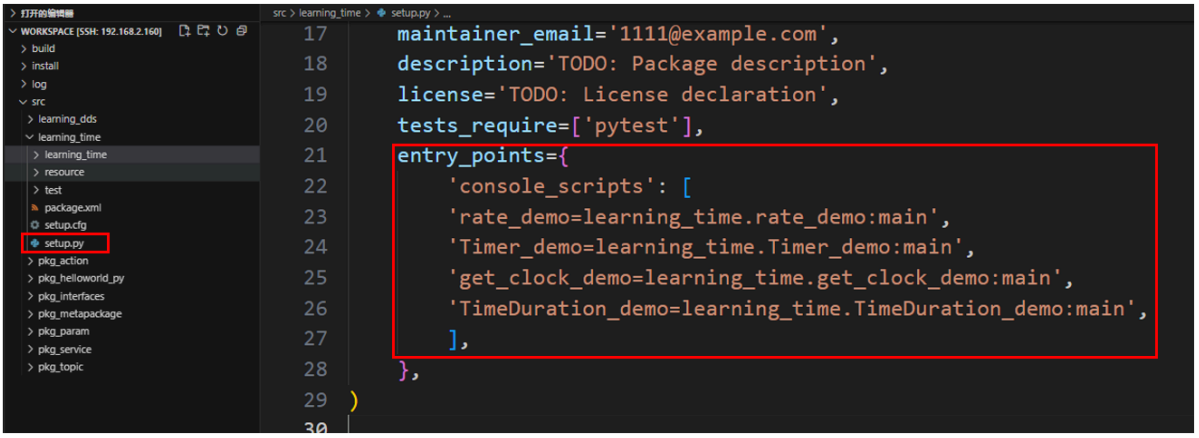
    # Time periods can be compared
    node.get_logger().info("-" * 80)
    node.get_logger().info("duration1 >= duration2 ? %d" % (duration1 >=
duration2))
    node.get_logger().info("duration1 < duration2 ? %d" % (duration1 <
duration2))

    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

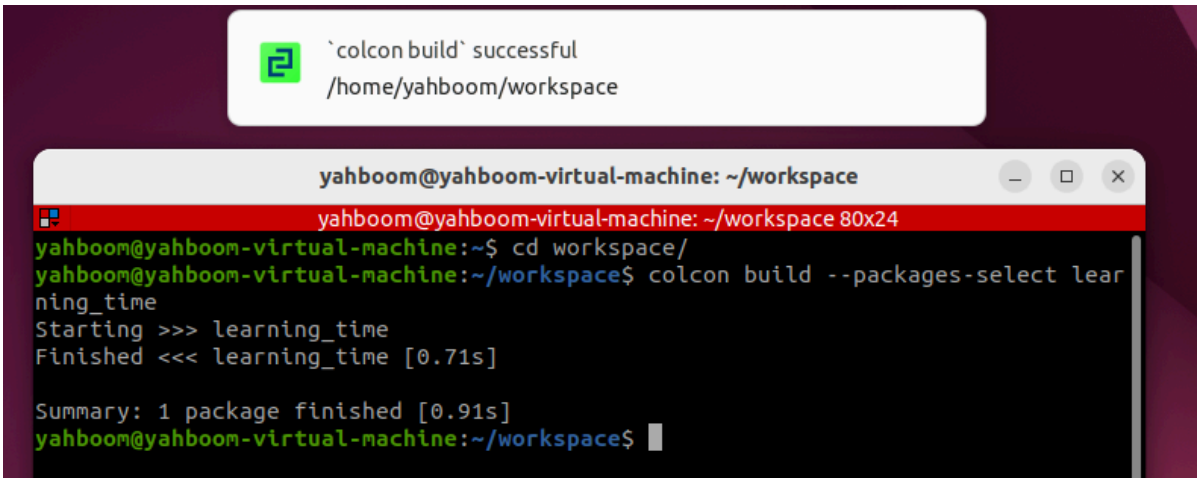
- Configure the corresponding setup.py configuration file, and add the following to console_scripts

```
'TimeDuration_demo=learning_time.TimeDuration_demo:main'
```




- Compile the package

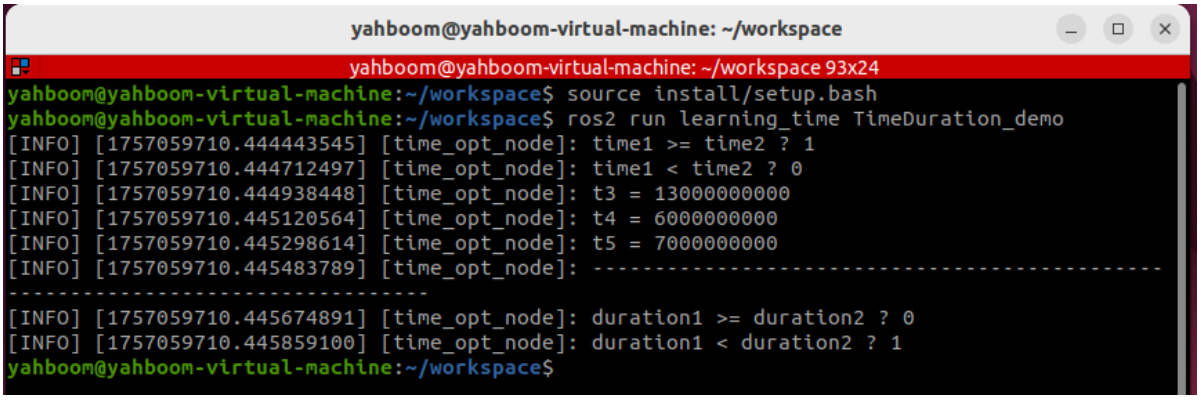
```
colcon build --packages-select learning_time
```




- Refresh the workspace environment and run the node

```
source ./install/setup.bash
```

```
ros2 run learning_time TimeDuration_demo
```




- This example proves that mathematical operations can be performed on time points and time periods, allowing us to flexibly query and operate on data of different timestamps